

3. Closure & Clones: Post's Lattice for $\mathbf{Bool}$

This section develops the Boolean case of closure theory in a form that is both algebraic and algorithmic. The central objects are *Boolean operations* $f: \mathbb{B}^n \rightarrow \mathbb{B}$, where $\mathbb{B} = \{0, 1\}$, together with the closure operators they generate under composition, permutation of variables, and insertion of projections. In the Boolean setting, these closure phenomena are organized by *clones*, and the global structure of all clones on $\mathbb{B}$ is described by Post's lattice.

A useful way to compare Boolean functions in this book is via a multi-resolution error metric. For two functions $f, g: \mathbb{B}^n \rightarrow \mathbb{B}$, one may measure disagreement by the Hamming distance of their truth tables, and then add a penalty when the functions do not use the same variable set or arity profile. We write this schematically as

$$\varepsilon(r)(f, g) = d_{\text{Ham}}(\text{tt}(f), \text{tt}(g)) + \lambda \pi(f, g),$$

where $\text{tt}(f)$ denotes the truth table of f , d_{Ham} is the Hamming distance on truth tables, $\pi(f, g)$ is a mismatch penalty for variable or arity differences, and $\lambda \geq 0$ is a weighting parameter. The exact choice of penalty is application-dependent, but the guiding principle is stable: semantic disagreement should be measured not only by output bits, but also by structural mismatch in the representation.

Closure operators and projections. A closure operator on a set of Boolean functions is a map C satisfying extensivity, monotonicity, and idempotence. In the present context, the closure of a set F of Boolean functions is the smallest class containing F that is closed under composition and under the insertion of projections. The projections are the coordinate maps

$$\pi_i^n(x_1, \dots, x_n) = x_i, \quad 1 \leq i \leq n,$$

and they play the role of the identity morphisms for function composition. Closure under projections ensures that variables may be selected, duplicated, or ignored as needed when building larger Boolean expressions from smaller ones.

Clones. A *clone* is a set of finitary operations on $\mathbb{B}$ that contains all projections and is closed under composition. Equivalently, a clone is a compositional universe of Boolean operations stable under substitution of operations into operations. Clones are the natural algebraic objects for describing what can be built from a given collection of gates or primitives. If F is a set of Boolean functions, then the clone generated by F , denoted $\langle F \rangle$, is the smallest clone containing F .

This perspective makes functional synthesis precise. Given a target Boolean function h , the question is whether $h \in \langle F \rangle$. If so, then h can be realized by a finite composition of functions from F and projections. If not, then F is insufficient for the desired expressive power, regardless of how deeply one composes its members.

Precomplete classes. A clone is called *precomplete* or *maximal proper* if it is a proper subclone of the full clone of all Boolean functions, but adding any function outside it yields the full clone. Precomplete classes mark the immediate boundary between incomplete and functionally complete systems. In hardware terms, they identify the last expressive classes before universality is obtained; in algebraic terms, they are the coatoms of the clone lattice.

Post’s lattice. Post’s lattice is the complete classification of all clones on $\mathbb{B}$. It organizes Boolean clones into a finite lattice of structurally significant families and their intersections. Among the most important named classes are the following:

- M : the monotone clone, consisting of functions preserving the pointwise order on $\mathbb{B}^n$;
- L : the affine clone, consisting of functions representable as affine polynomials over $\mathbb{F}_2$;
- D : the self-dual clone, consisting of functions satisfying $f(\bar{x}) = \overline{f(x)}$;
- T_0 : the clone of functions preserving 0;
- T_1 : the clone of functions preserving 1.

These classes are not isolated curiosities; they are the principal axes along which Boolean expressivity decomposes. For example, monotonicity constrains the direction of information flow, affine structure constrains parity behavior, and self-duality encodes a symmetry under bit-flip complementation.

The lattice viewpoint is especially useful because it turns expressive power into order-theoretic inclusion. If $C_1 \subseteq C_2$, then every function definable in C_1 is also definable in C_2 . Thus, moving upward in Post’s lattice corresponds to gaining expressive power, while moving downward corresponds to imposing stronger invariants.

Functional completeness. A set of Boolean functions is *functionally complete* if it generates the full clone of all Boolean functions on $\mathbb{B}$. Classical criteria due to Sheffer characterize when a single binary connective suffices. In particular, NAND and NOR are functionally complete: each alone generates all Boolean functions. More generally, a set of functions is complete precisely when it is not contained in any of the maximal proper clones identified by Post’s classification. This yields a decision procedure: to test completeness, it suffices to check whether the generating set lies inside one of the precomplete classes.

In practice, this criterion is often used in two directions. First, it certifies universality of a gate set. Second, it proves impossibility: if a candidate library is contained in M , for example, then it cannot express negation and therefore cannot be complete. Similar obstructions arise for L , D , T_0 , and T_1 .

Decision procedures and algorithms. The classification of Boolean clones supports concrete algorithms for synthesis and verification. A typical workflow is as follows:

1. normalize the candidate functions by truth table or algebraic normal form;
2. test membership in the principal Post classes M , L , D , T_0 , and T_1 ;
3. infer the smallest clone containing the set, or at least a tight upper bound on it;
4. determine whether the set escapes every maximal proper clone, hence is functionally complete.

These tests can be implemented using truth-table inspection, polynomial-time checks for the named classes, and search over canonical representatives of clone families.

A related computational problem is clone membership: given a target function h and a generating set F , decide whether $h \in \langle F \rangle$. For small arities, brute-force enumeration of the closure may be feasible; for larger instances, one typically relies on structural invariants, normal forms, or reductions to known clone classes. The book will later reuse these ideas in symmetry analysis and in algorithmic closure checking.

NPN classes and canonicalization. Boolean functions are often compared up to negation, permutation, and input negation, the so-called NPN transformations. These equivalence classes are useful for reducing redundancy in search and for identifying canonical representatives of gate libraries. While Post’s lattice classifies closure under composition, NPN classification organizes functions by symmetry. The two viewpoints complement each other: NPN equivalence simplifies the search space, while clone membership determines expressive closure.

Minimal generating sets. A generating set for a clone is minimal if no proper subset generates the same clone. Minimal generating sets are important both theoretically and practically. Theoretically, they expose the irredundant primitives underlying a class of Boolean behavior. Practically, they identify compact gate libraries and reduce synthesis complexity. In the Boolean setting, minimality is often studied together with canonical forms and with the precomplete classes of Post’s lattice.

The overarching message is that Boolean expressivity is not a flat collection of gates but a structured hierarchy of closure classes. Projections provide the base level of composition, clones capture substitutional closure, Post’s lattice classifies the resulting universe, and functional completeness identifies the boundary where a gate set becomes universal. This algebraic picture will serve as the foundation for later chapters on modular circuit construction, symmetry analysis, and algorithmic verification.